마이크로프로세서

Chapter 1. Introduction to STM32 Processors

Human-Robot Interaction Laboratory

KYUNG HEE UNIVERSITY

ARM Processor?

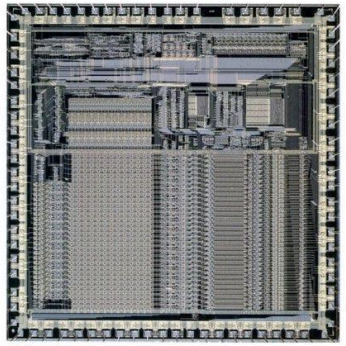
>> History of ARM Processor

- 1983년 영국의 Acorn Computer가 저전력 고성능 프로세서 개발 프로젝트(Acorn RISC Machine) 시작

- 1990년대 Acorn Computer, VLSI Technology, Apple Computer 합작회사 ARM Ltd. 설립

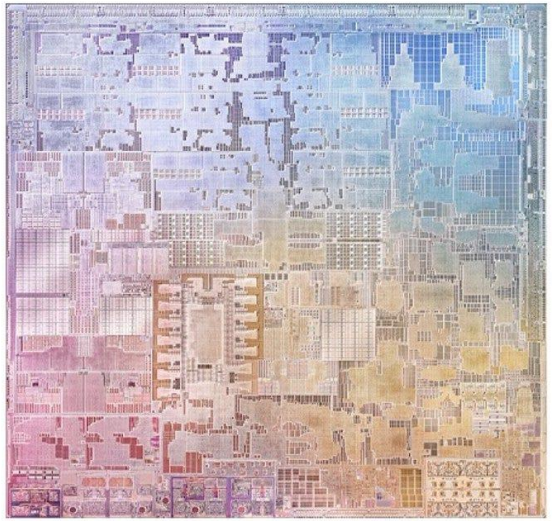
- 이후 ARM7 아키텍쳐의 모바일 폰 시장에서의 성공과 Apple의 iPhone에 ARM이 탑재되며 점유율 증가

ARM1 and Apple M1 at the same scale

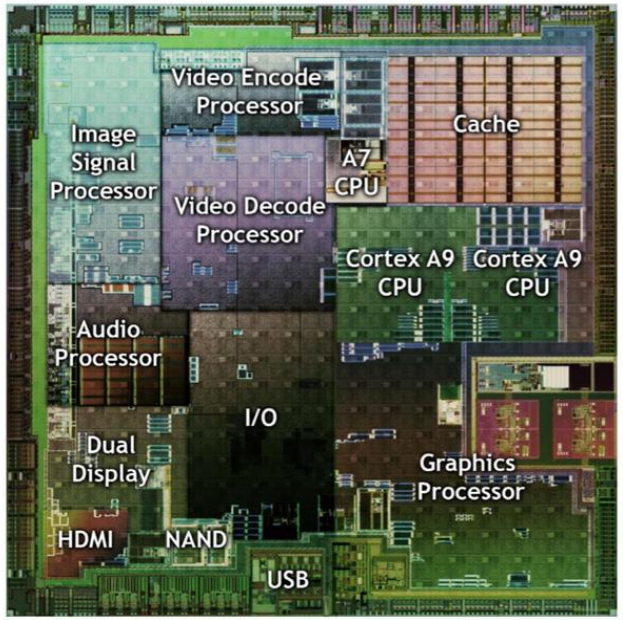


ARM1 processor
~7mm wide
3 µm process

If built with M1 transistors, the entire ARM1 chip would be the tiny dot inside this circle:



Apple M1 processor, ~11mm wide, 5 nm process



KYUNG HEE UNIVERSITY

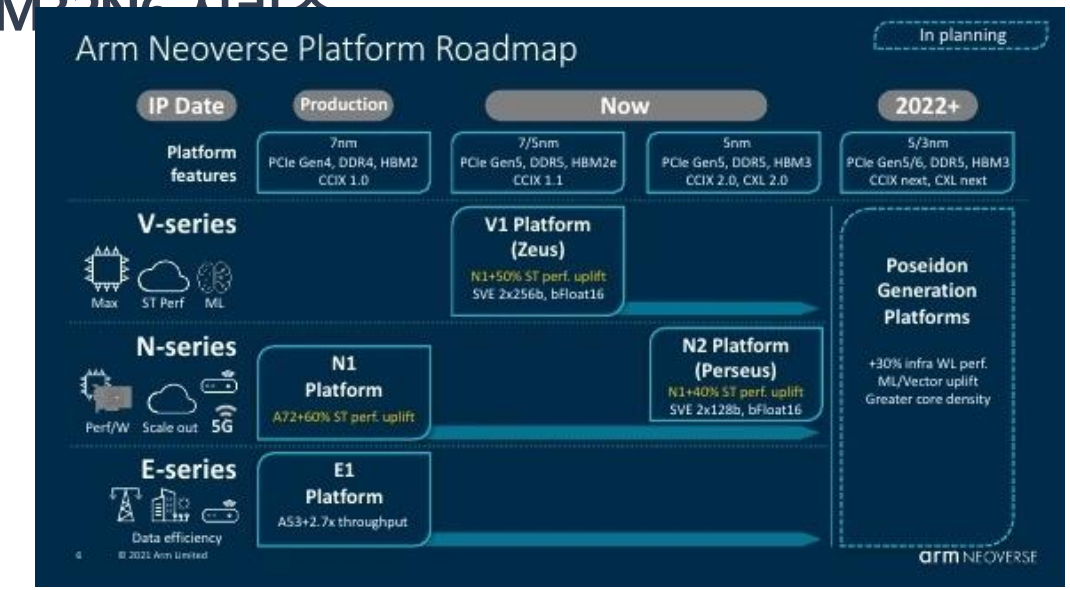
ARM Processor?

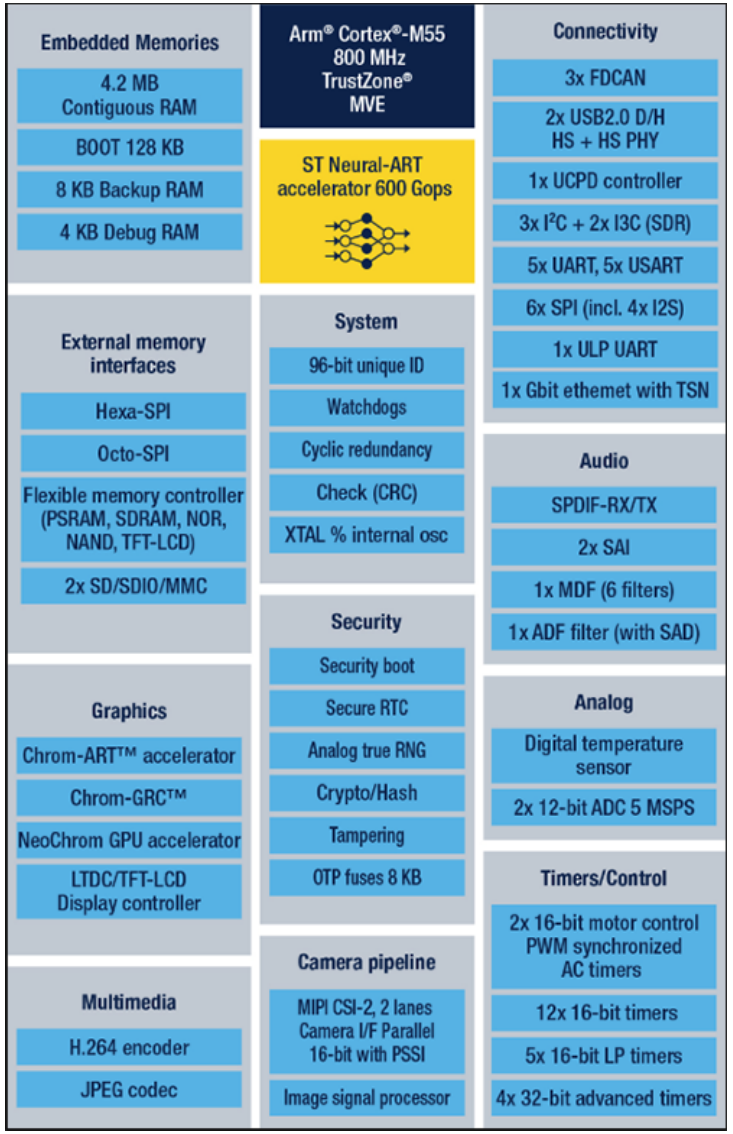
>> History of ARM Processor

- 현재 스마트폰의 99%는 ARM을 활용

- 자동차, 모바일 기기를 넘어 데이터 센터 및 클라우드 서버 제품군 까지 확장

- 2024년말 AI 시대를 위한 신경 처리 프로세서(Neural Processing Unit)를 탑재한 STM32N6 시리즈

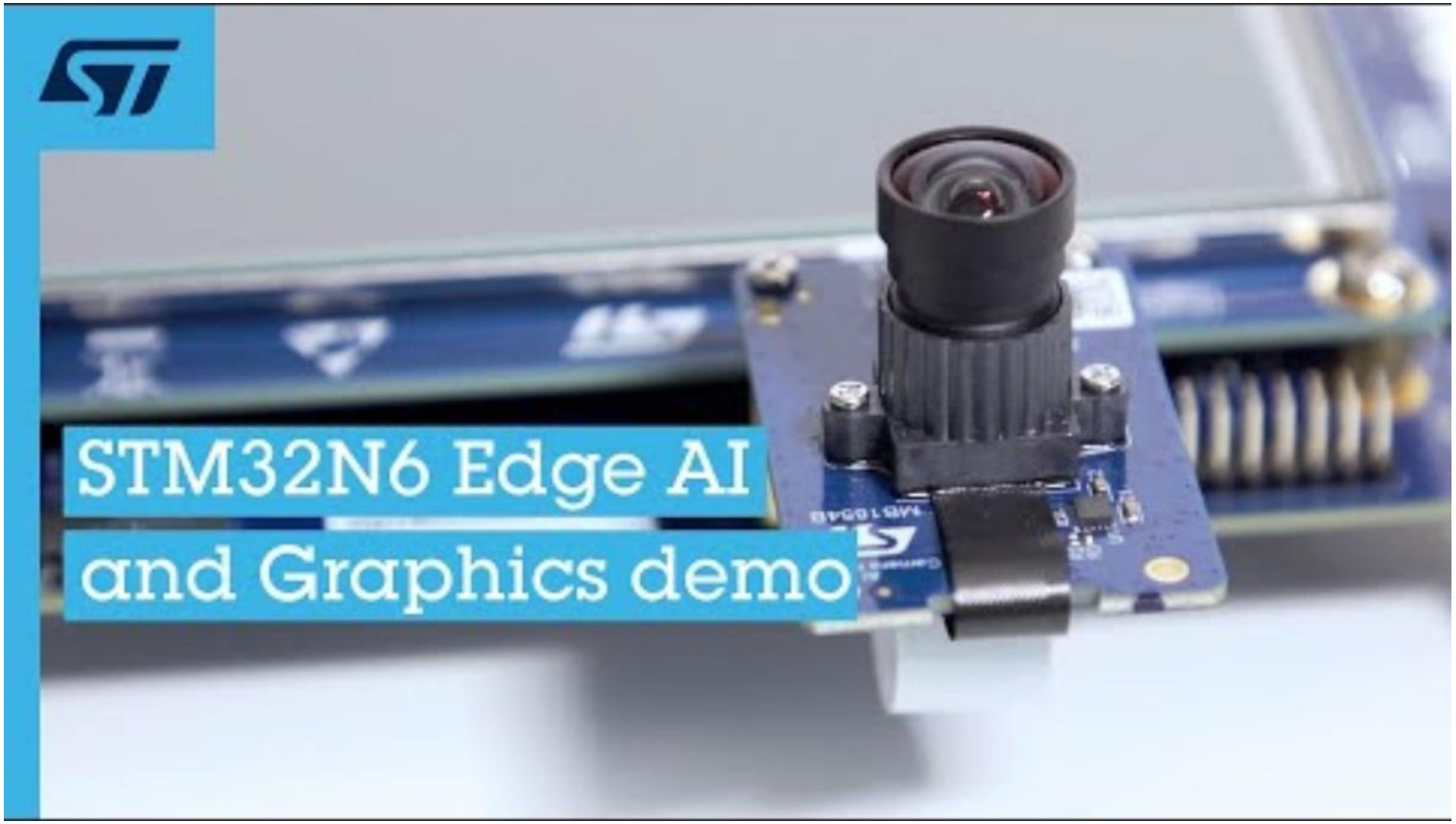




KYUNG HEE UNIVERSITY

ARM Processor?

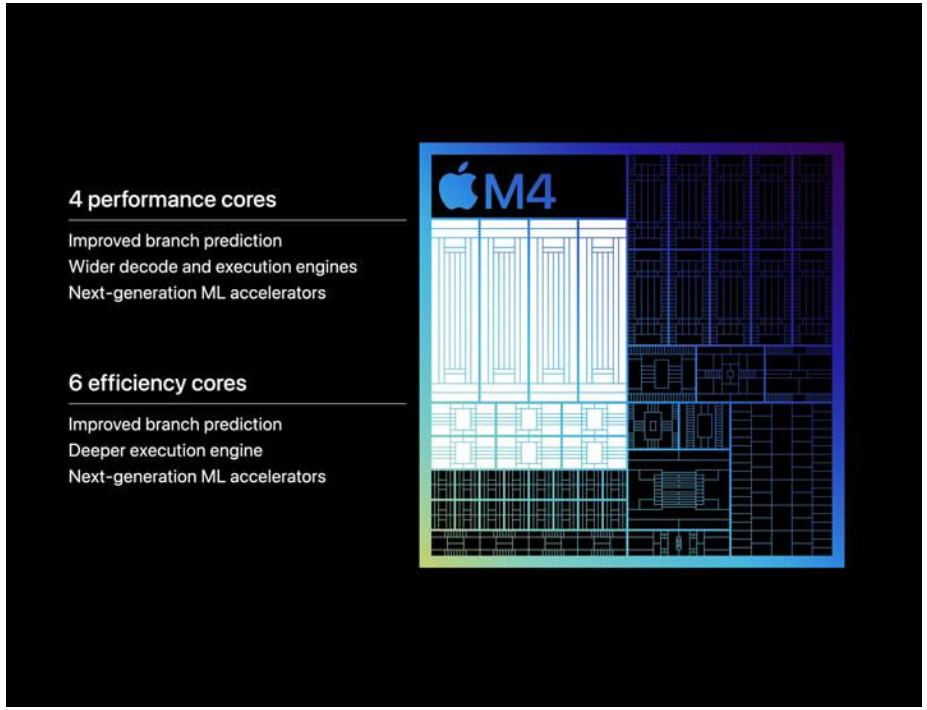
>> History of ARM Processor



ARM Processor?

>> Industry Applications and Market Trends of ARM Processor

- Apple Silicon M4
- ARMv9 기반의 최신 아키텍쳐
- 높은 수준의 프로세서 성능과 높은 전력효율이 특징



ARM Processor?

>> Industry Applications and Market Trends of ARM Processor

- Qualcomm Snapdragon
- Apple Silicon M 시리즈의 성공 이후 Arm 프로세서 기반의 노트북 시장이 발전
- Arm 윈도우 PC 시장의 성장과 AI 기능(Copilot)의 결합을 통한 시장 변화

KYUNG HEE UNIVERSITY

ARM Processor?

>> Industry Applications and Market Trends of ARM Processor

- AI 시장에서의 ARM기반 프로세서 입지 강화
- Edge AI를 활용한 실시간 AI 서비스, 자율주행, IoT 서비스 등 다양한 산업 분야의 발전

ARM "AI 기능으로 5년내 PC 시장 50% 점유할 것"

박찬 기자 입력 2024.06.04 18:00 댓글 0 좋아요 0

새 엣지 AI 시대 연다...Arm, 초고효율 CPU·플랫폼 공개

최초의 엣지 AI Armv9 플랫폼으로 스마트 카메라, 산업 자동화 등 지원

수정: 2025/02/27 14:29

홈 > 뉴스 > 인프라

AI PC 성능 가늠좌 'NPU', 칩메이커 격전지로 부상

남도영 기자 | 승인 2024.05.08 12:01

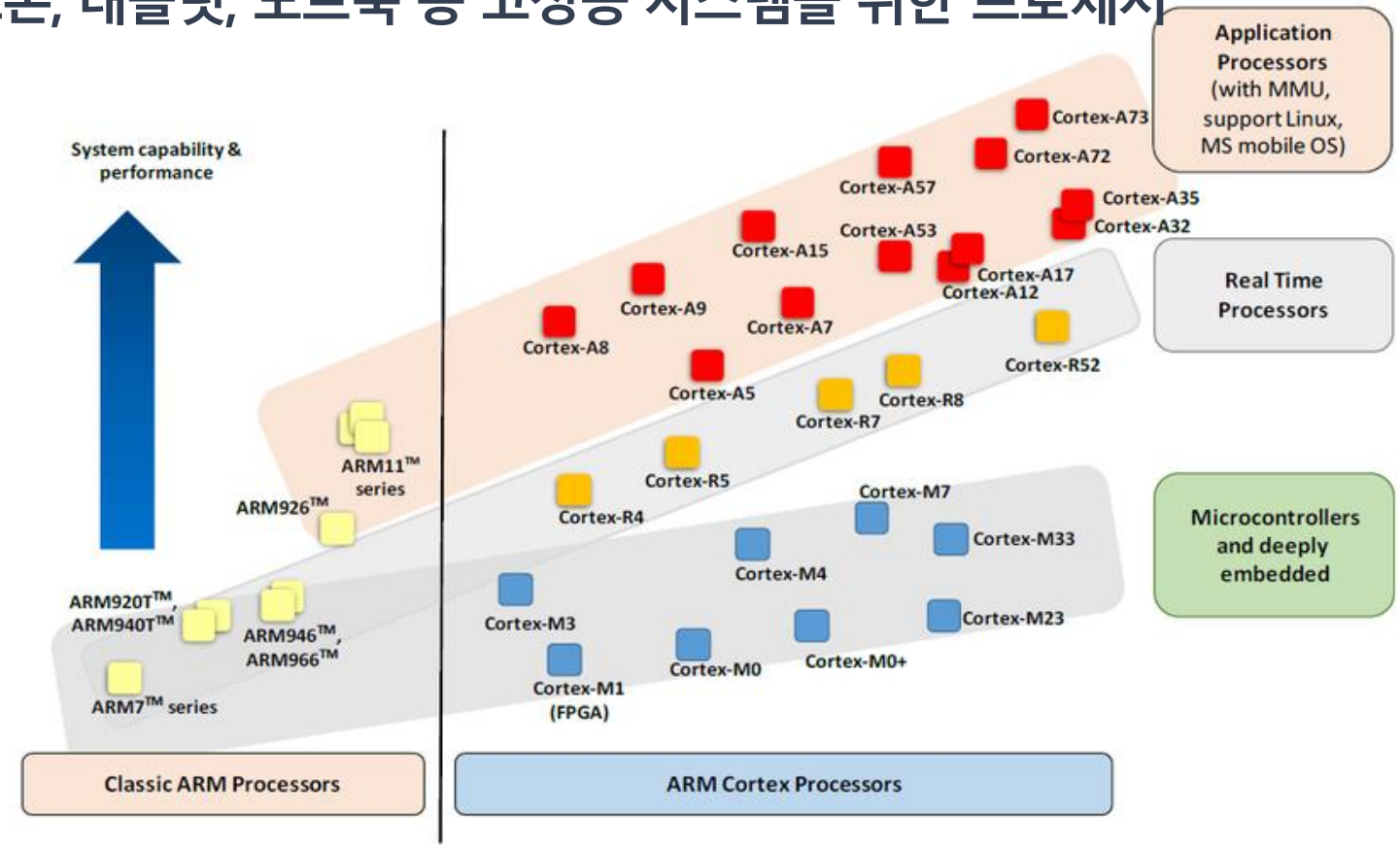
가 가

KYUNG HEE UNIVERSITY

ARM Architecture Family

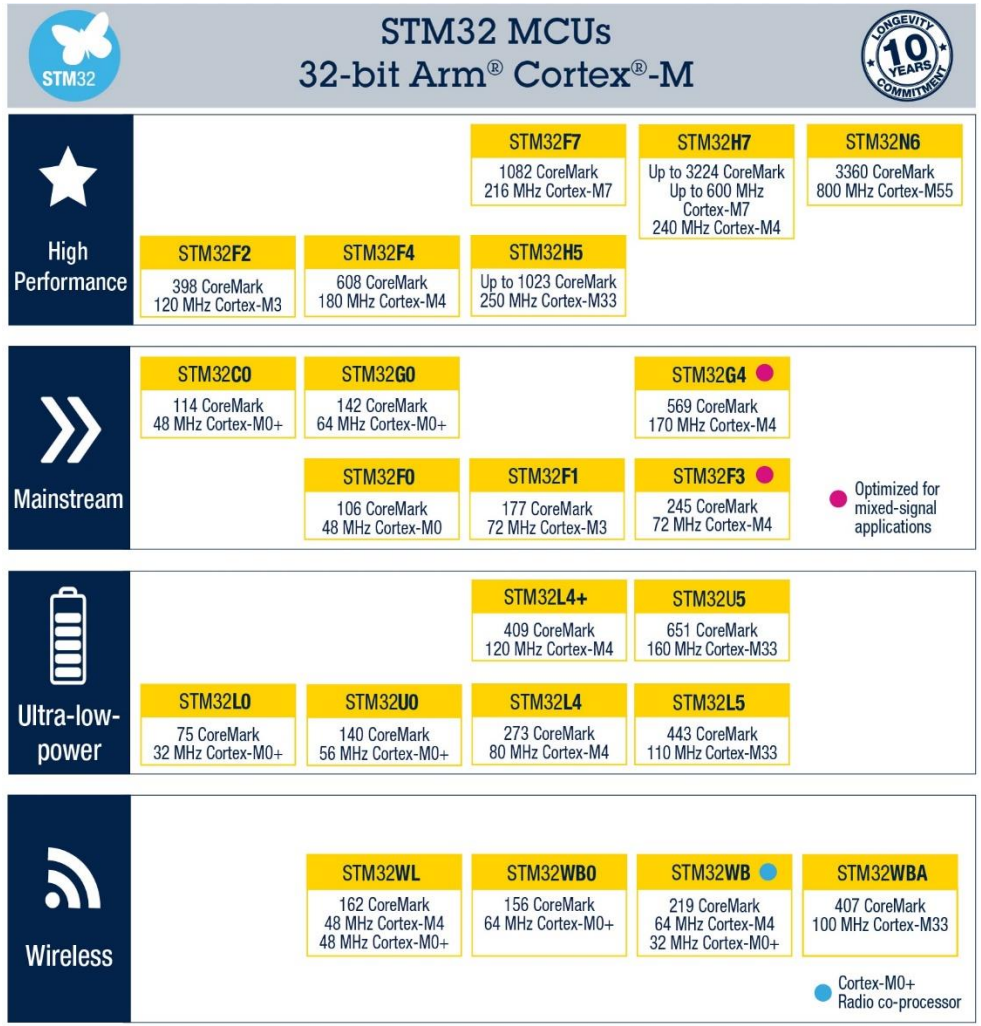
>> STM32 Cortex Family

- Cortex-M: 저전력, IoT, 임베디드에 활용되는 프로세서
- Cortex-R : 실시간, 안전성이 요구되는 자동차, 산업, 의료 분야를 위한 프로세서
- Cortex-A: 스마트폰, 태블릿, 노트북 등 고성능 시스템을 위한 프로세서

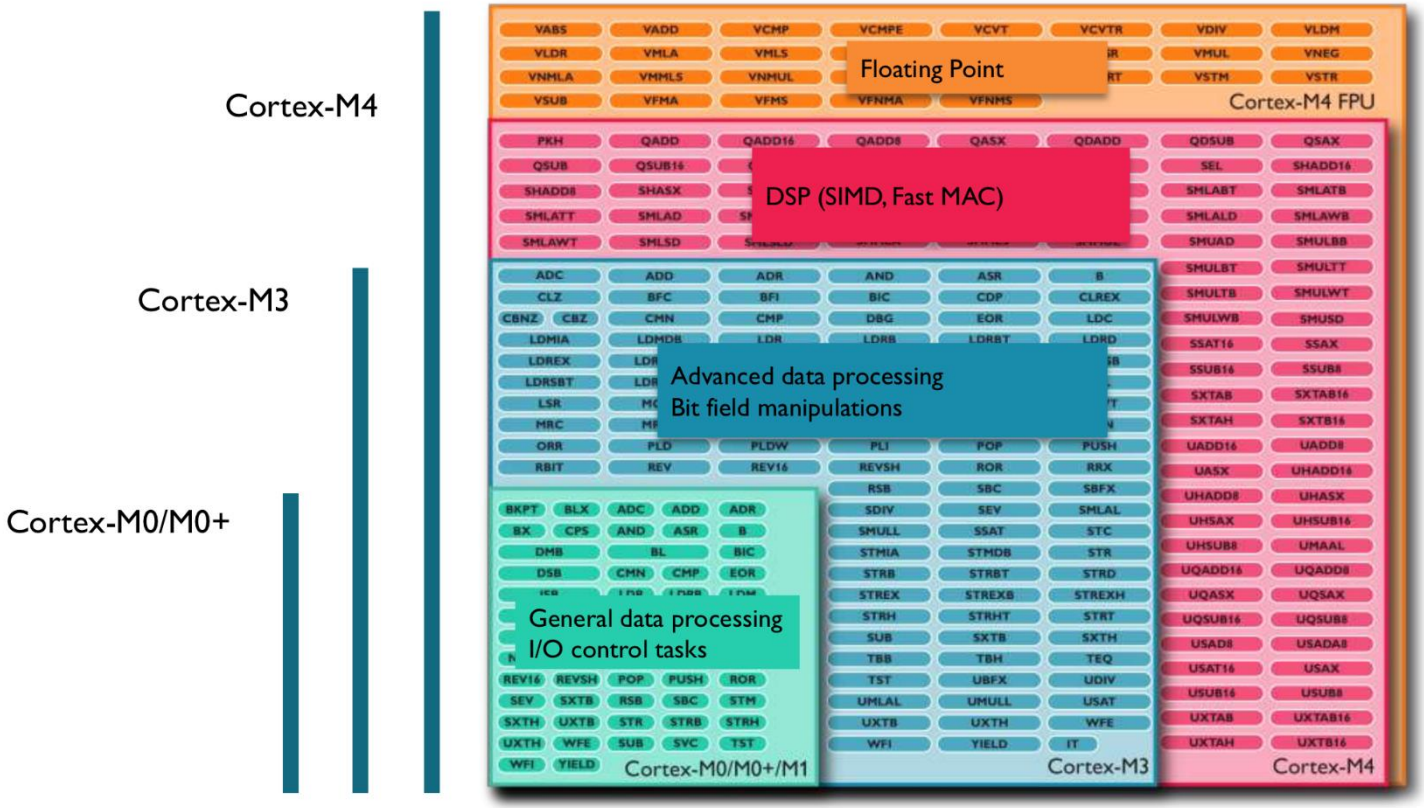


ARM Architecture Family

>> STM32 Cortex Family



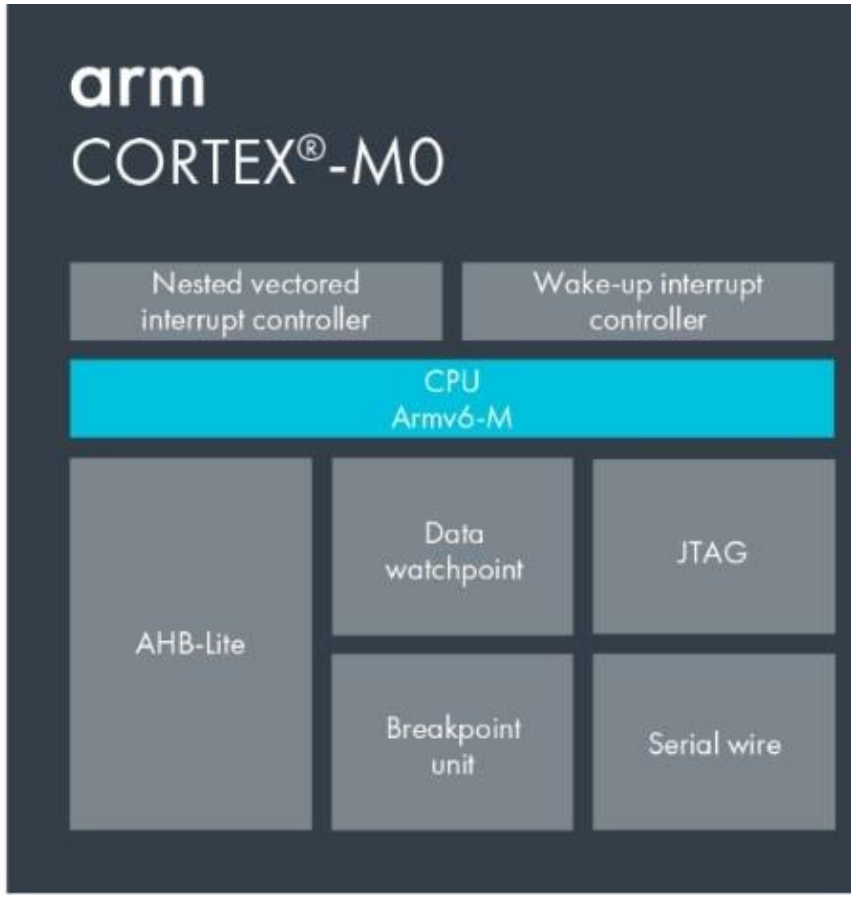
ARM Cortex-M Instruction Set Architecture

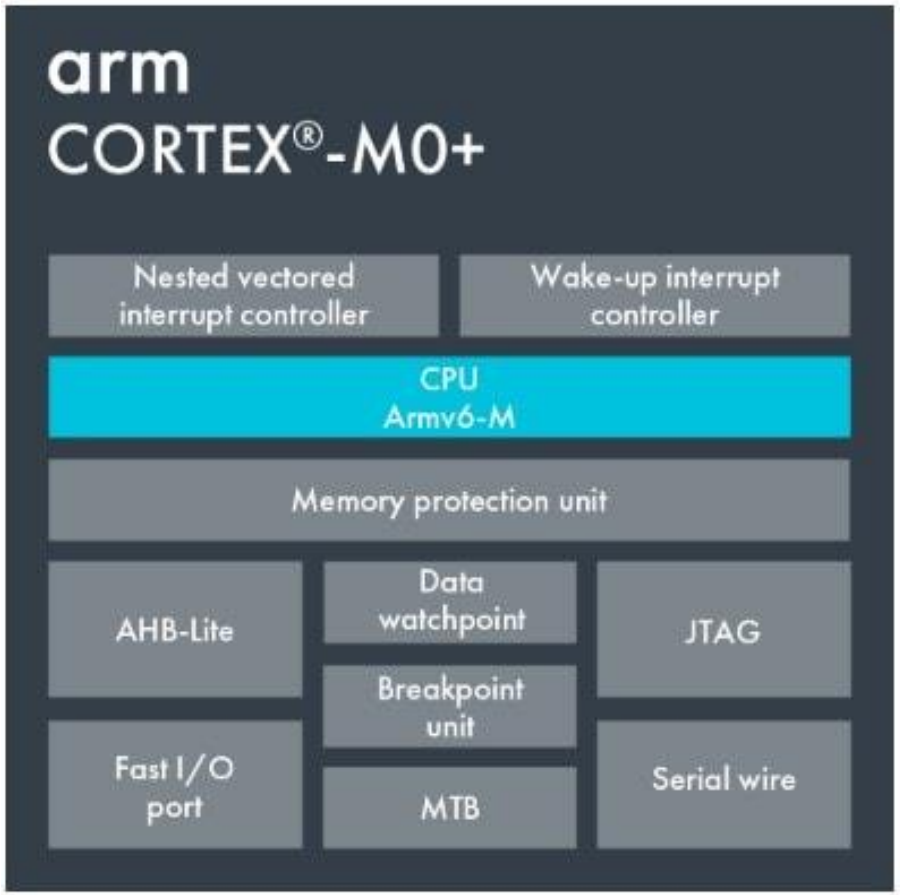


Cortex-M Family

>> Cortex-M0, Cortex-M0+

- 저전력 설계, 단순한 구조, 가장 저렴한 프로세서
- 간단한 제어 및 센서 데이터 수집, IoT 프로젝트에 적합

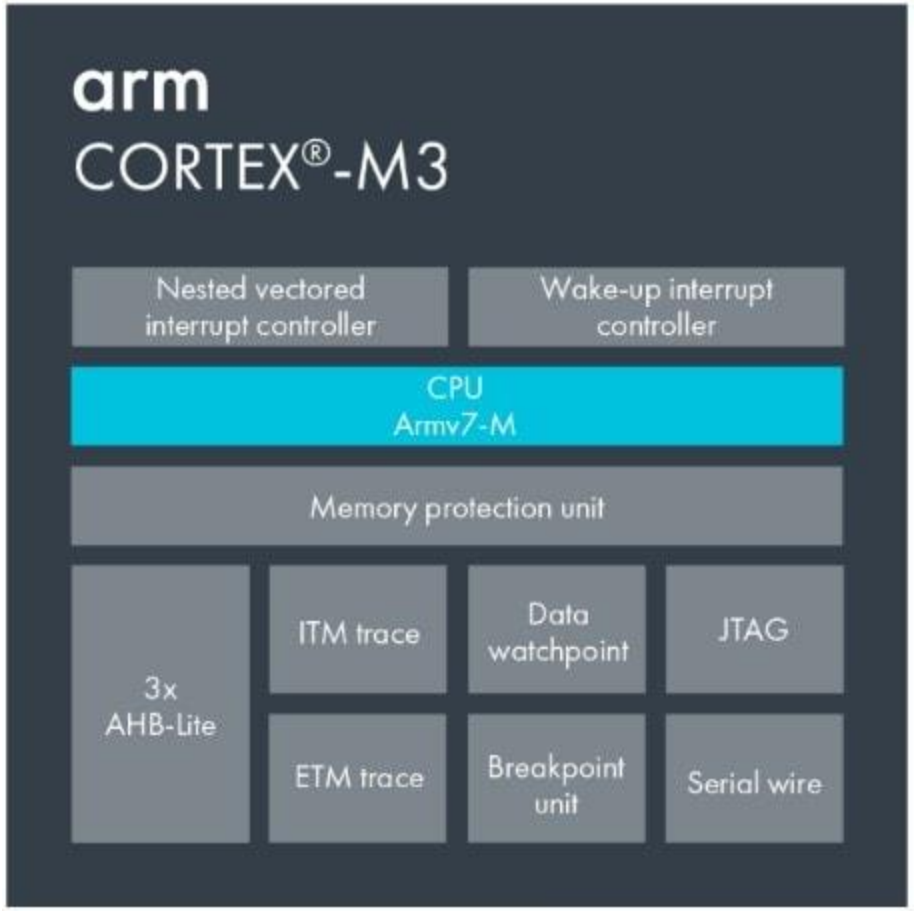




Cortex-M Family

>> Cortex-M3

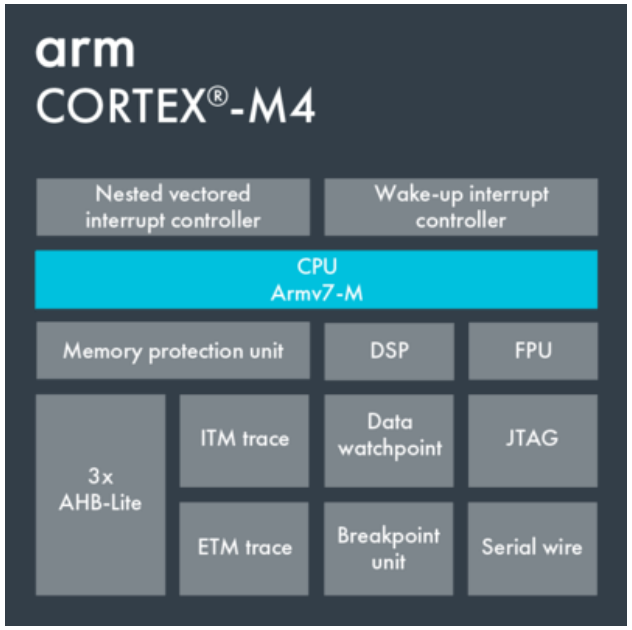
- Armv7-M 기반의 아키텍쳐
- Thumb, Thumb-2 ISA 지원
- 32비트 하드웨어 곱셈, 나눗셈 연산
- 최대 240개의 인터럽트 처리
- Nested Vectored Interrupt Controller, 우선순위에 따른 효율적인 인터럽트 처리
- 최대 8개의 메모리 보호 유닛(Memory Protection Unit)

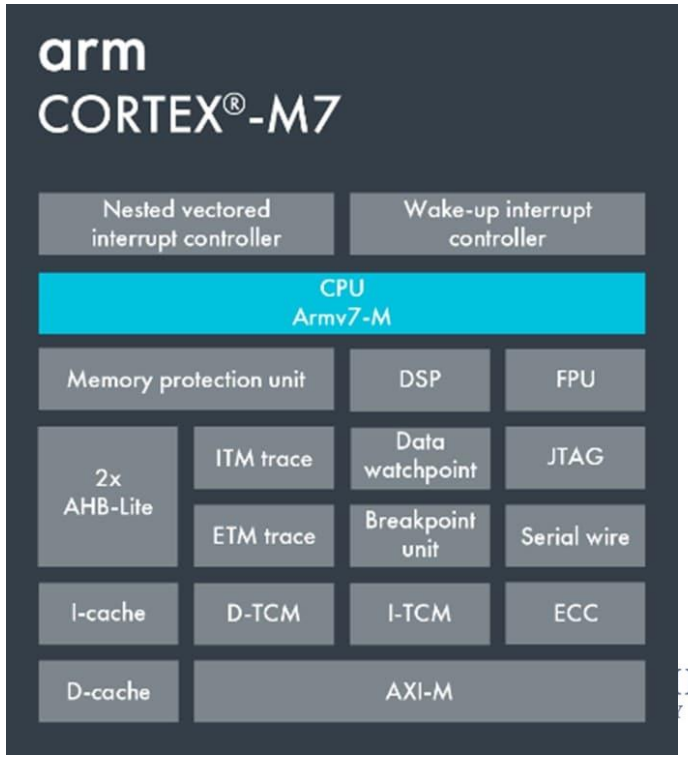


Cortex-M Family

>> Cortex-M4, Cortex-M7

- DSP(디지털 신호 처리), FPU(부동소수점 연산) 기능 추가됨
- M3에 비해 높은 연산 성능을 제공 하며, 연산과 신호/오디오 처리가 가능함
- M7은 고속 메모리(TCM), 메모리 오류 감지 및 수정(ECC) 기능 제공
- 차량, 항공기와 같은 고성능 제어, 실시간 제어가 필요한 시스템에 활용됨
- 오디오, 비디오, 이미지 처리등 고속 신호 처리 시스템에 활용 가능

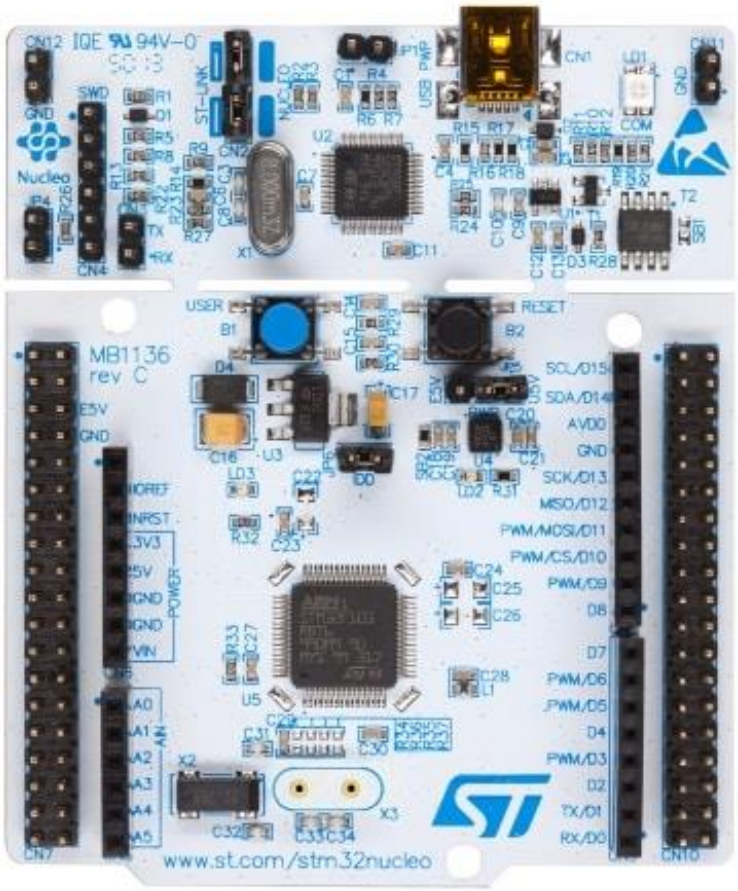


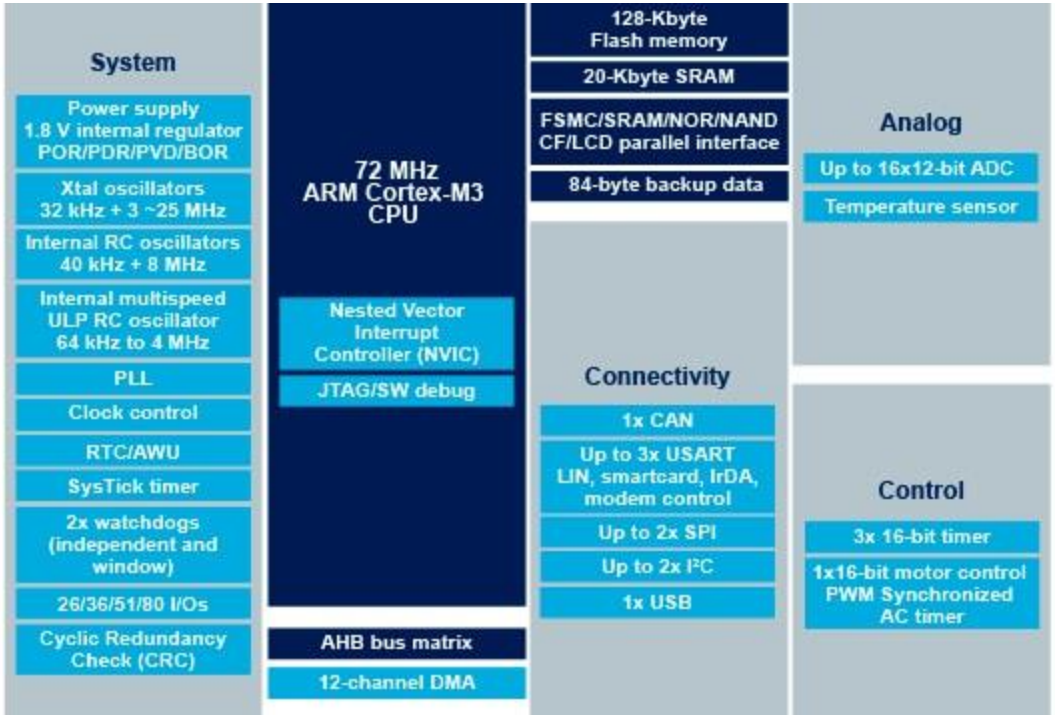


STM32 마이크로프로세서

>> STM32F103RB

- 강의에서 활용할 프로세서: STM32F103RB(Cortex-M3 기반)

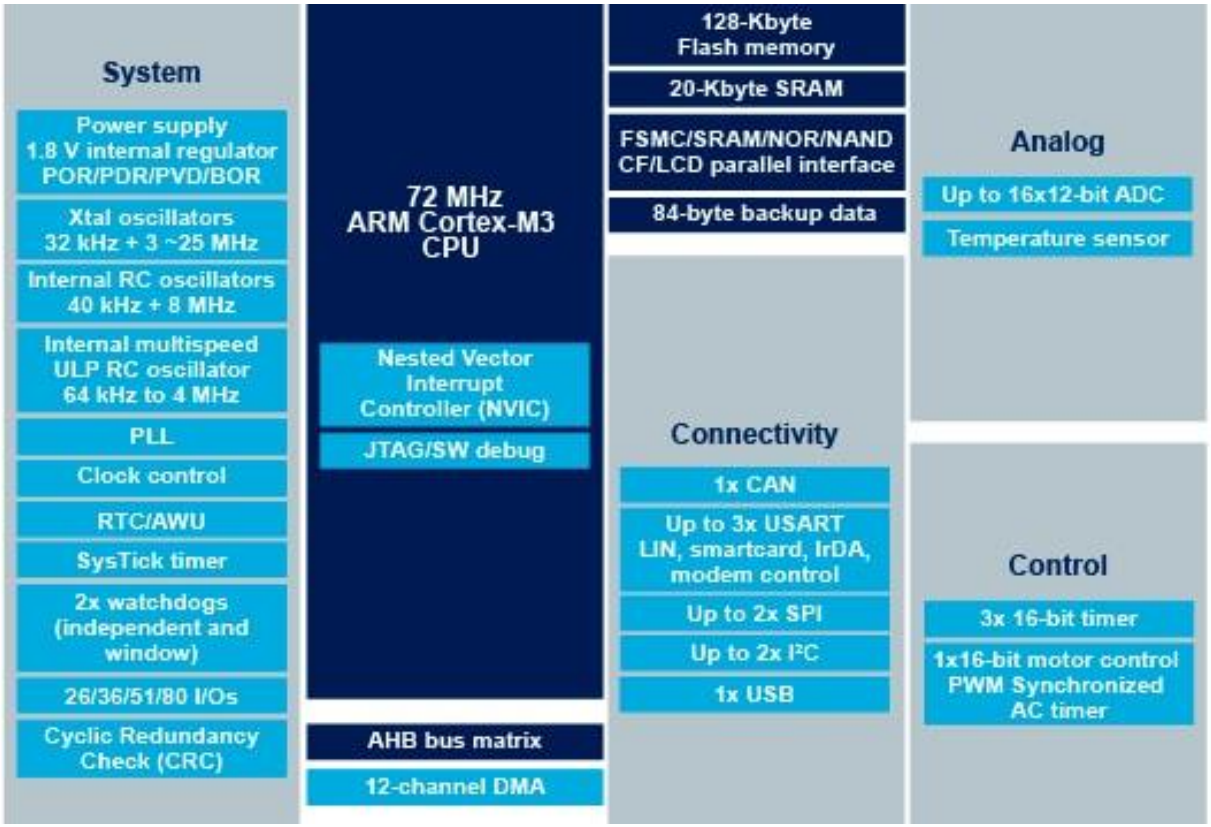




STM32F103RB

>> Overview

- STM32F103RB
  - Cortex-M3 CPU (~72MHz)
  - 128KB Flash, 20KB SRAM
  - Internal 8MHz, 40kHz RC oscillators
  - Phase-Locked Loop for CPU clock
  - 2x 12bit ADC (up to 16ch)
  - 7ch Direct Memory Access
  - 7x 16bit Timer
  - I2C, USART, SPI, CAN, USB Connectivity
  - JTAG/SWD 디버깅 지원



STM32F103RB

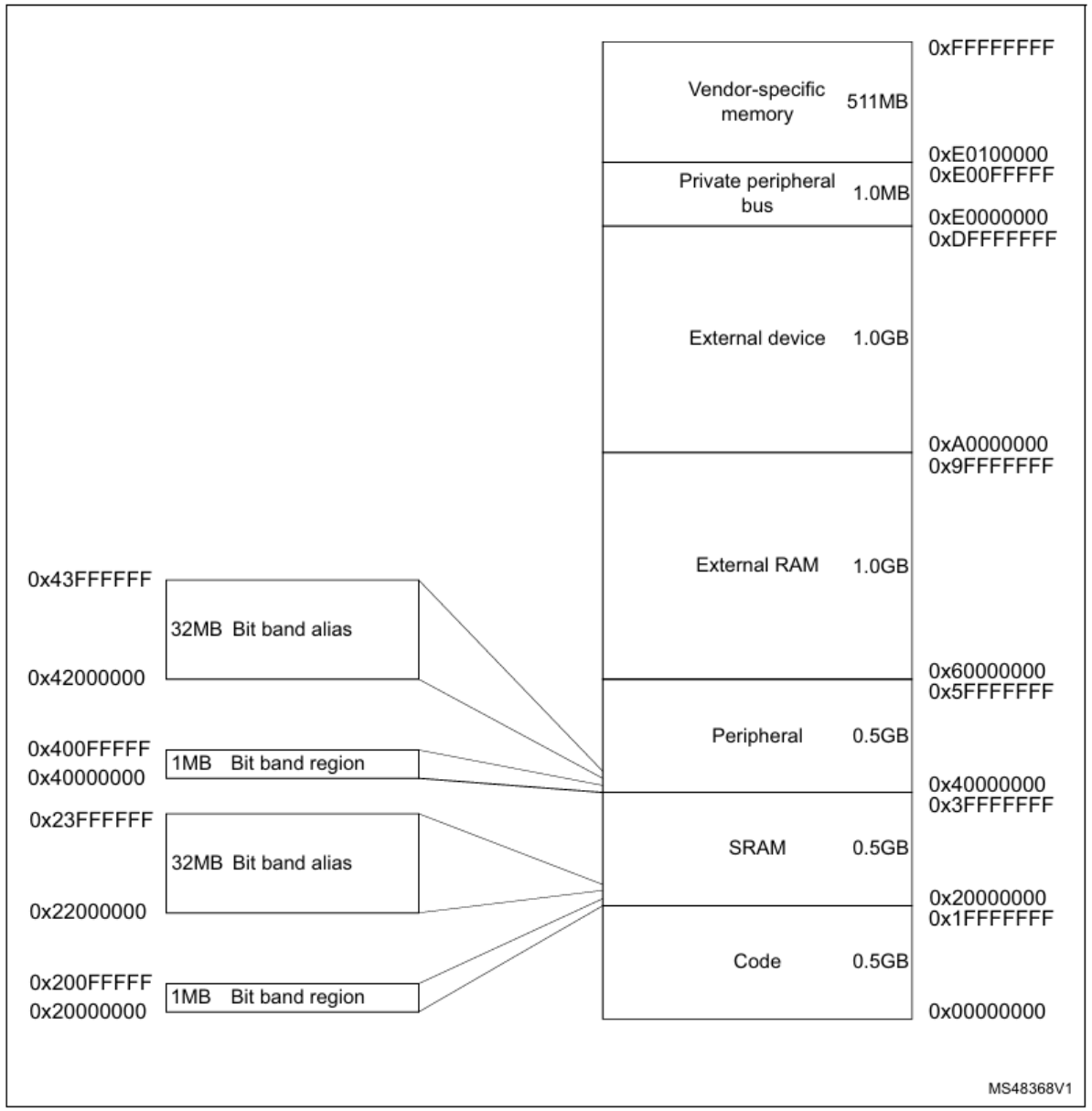
>> Memory Map

- 32bit 프로세서로 0x00000000 ~ 0xFFFFFFFF 총 4GB의 메모리 공간으로 구성됨

$$2^{32} = 4,294,967,296\ (4GB)$$

- Code Region
  - 0x00000000 ~ 0x1FFFFFFF (512MB)
  - Flash 메모리 영역
- SRAM
  - 0x20000000 ~ 0x3FFFFFFF (512MB)
  - 스택, 힙, 전역 변수 등 런타임 데이터
- Peripheral Region
  - 0x40000000 ~ 0x5FFFFFFF (512MB)
  - GPIO, USART, SPI등 주변 장치 레지스터

Figure 9. Memory map



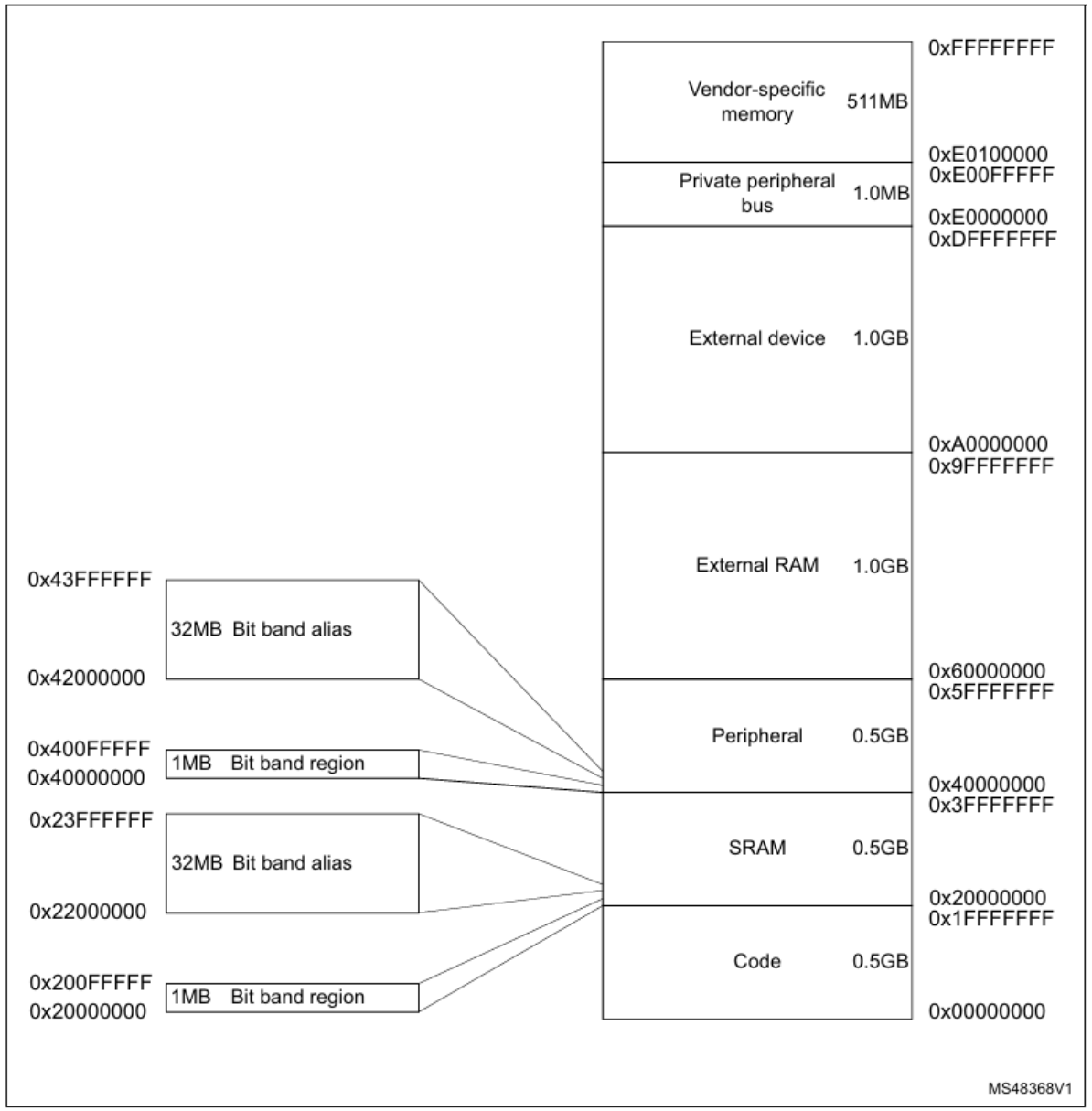
KYUNG HEE UNIVERSITY

STM32F103RB

>> Memory Map

- **External RAM Region**
  - 0x60000000 ~ 0x9FFFFFFF (1GB)
  - 외부 SRAM, SDRAM 메모리 연결 가능
- **External Device Region**
  - 0xA0000000 ~ 0xDFFFFFFF (1GB)
  - 외부 장치, 메모리 매핑 I/O 장치 연결 가능
- **Private Peripheral Bus, PPB**
  - 0xE0000000 ~ 0xE00FFFFF (1MB)
  - 시스템 제어 및 상태 정보 관리 레지스터 영역
  - Nested Vectored Interrupt Controller(NVIC), 시스템 타이머등 인터럽트 관리와 작업 우선 순위 처리를 관리
- **Vender-specific Memory**
  - 0xE0100000 ~ 0xFFFFFFFF (511MB)
  - 제조사별 추가 시스템, 디버깅 기능 영역

Figure 9. Memory map



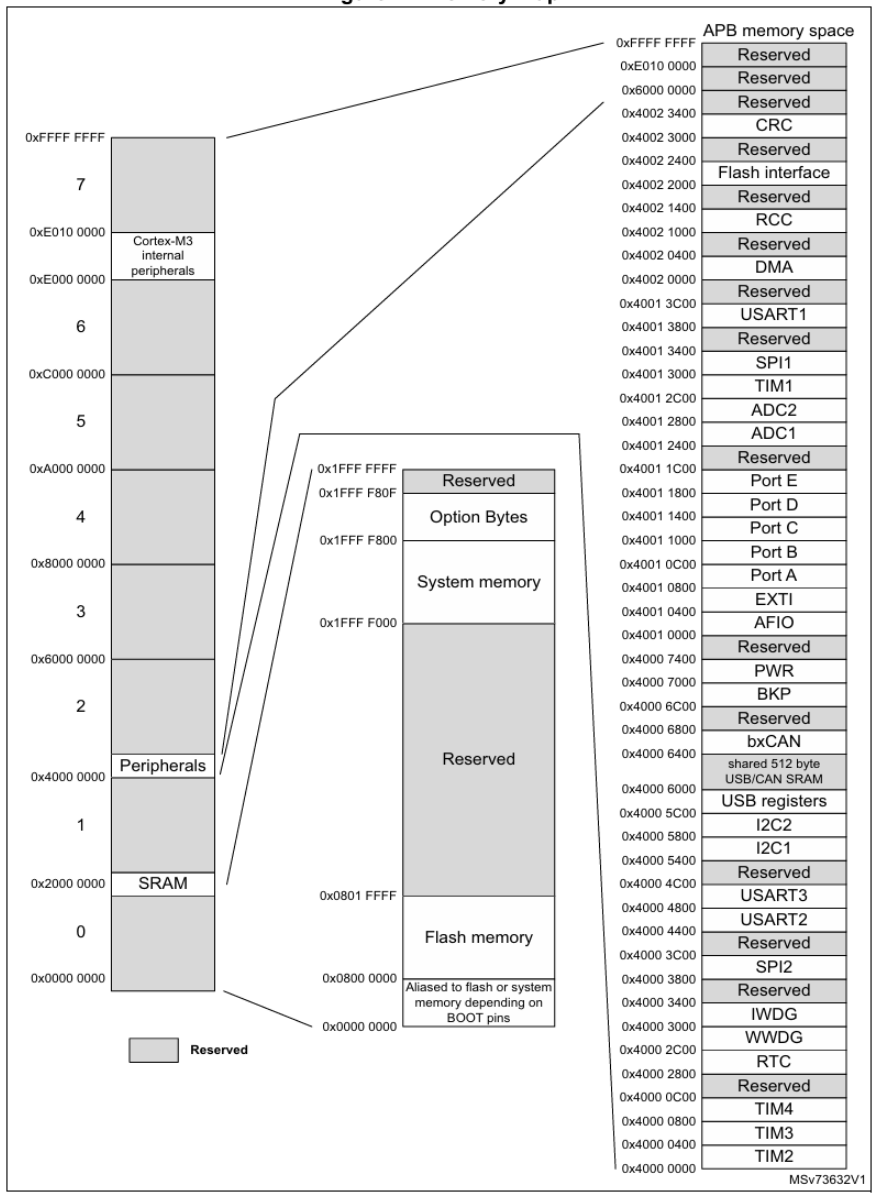
KYUNG HEE UNIVERSITY

STM32F103RB

>> Memory Map

- 프로세서는 코드 영역에 저장되고 실행됨(Flash memory)
- 전원이 인가되면 리셋을 통해 프로세서 초기화를 진행
- 시작 코드 실행, 상수, 변수 초기화 및 할당 진행
- 사용자 코드인 main() 함수 호출
- 동작 과정에서 SRAM은 스택, 힙 메모리 영역으로 활용됨
  - 스택 메모리: 코드 실행 과정에서 지역 변수, 매개변수, 반환값, 함수 호출등의 과정에서 할당 및 반환
  - 힙 메모리: 동적 할당, 사용자가 반환 시점을 지정
- GPIO, UART, PWM등 다양한 Peripheral 동작은 Peripheral Region의 여러 레지스터를 조정함으로써 활용

Figure 11. Memory map



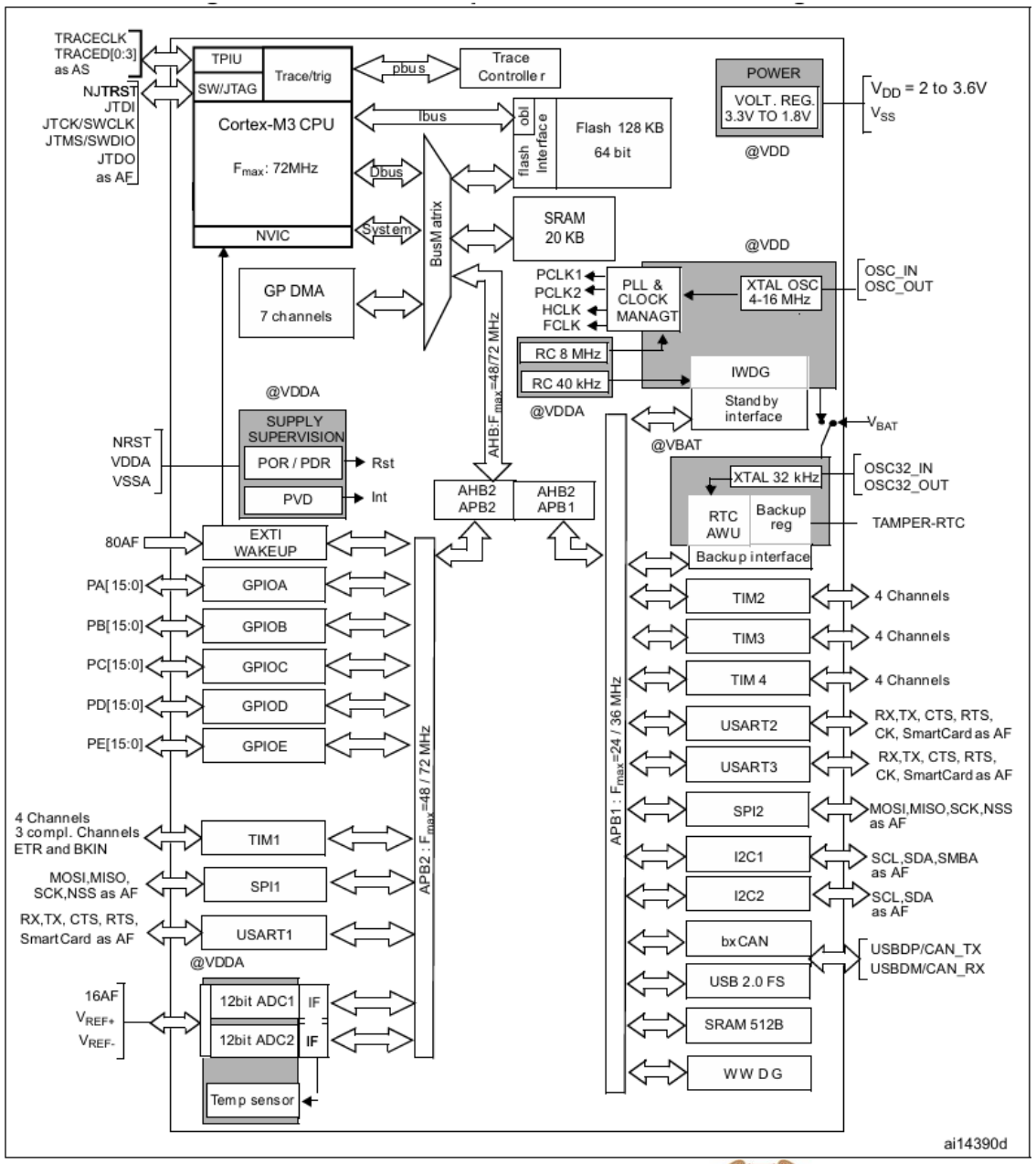
KYUNG HEE UNIVERSITY

STM32F103RB

>> Structure

- **NVIC**
  - Nested Vectored Interrupt Controller, 중첩 벡터 인터럽트 컨트롤러
  - 시스템의 인터럽트, 예외처리를 Interrupt Vector Table을 참조하여 작업의 우선순위를 결정함
  - Tail-Chaining: 연속적인 인터럽트 발생시 선행 인터럽트 종료와 후속 인터럽트 시작 사이의 스택 작업(Pop-Push)을 생략하여 빠른 인터럽트간 지연(Cyde)을 최소화
  - Late Arrival: 인터럽트 처리(Push) 도중 우선순위가 더 높은 인터럽트 발생시 즉시 새 인터럽트를 추가(연속으로 Push)하고 Tail-Chaining을 통해 지연(Cyde)을 최소화

- **DMA**
  - Direct Memory Access, CPU의 개입 없이 I/O 장치와 메모리가 직접 데이터를 전송하는 기능
  - CPU의 부하를 줄이고, 데이터 전송 효율을 높히는 기능



KYUNG HEE UNIVERSITY

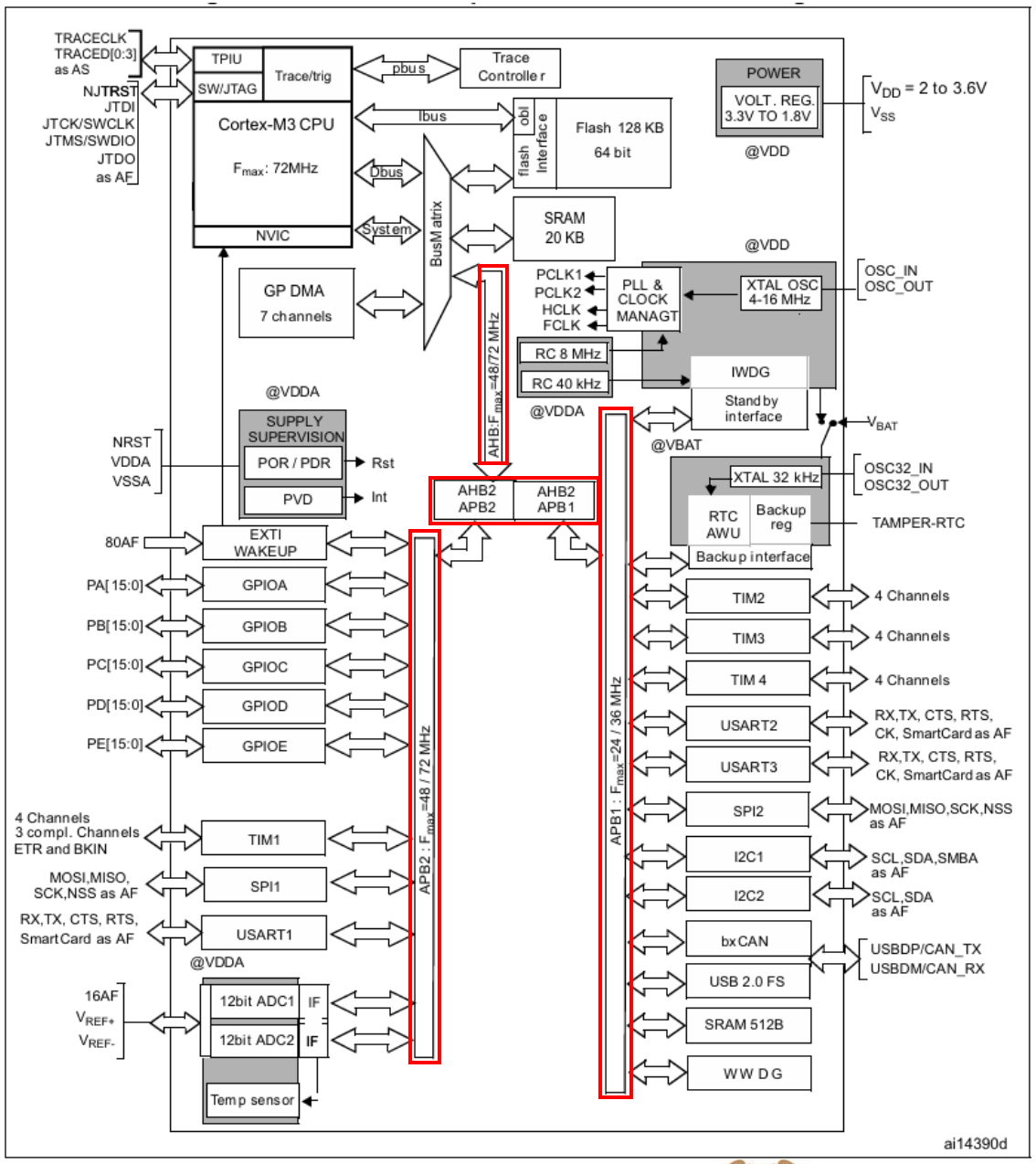
STM32F103RB

>> Structure

- **AHB/APB**
  - Advanced High-Performance Bus, Advanced Peripheral bus
  - AHB: 넓은 대역폭의 고속 데이터 전송 버스, 메모리와 DMA 컨트롤러, CPU 사이의 고속 데이터 통신 지원
  - APB: 저속 주변 장치(GPIO, UART, SPI, I2C 등) 사이의 통신 지원, 단순한 설계와 낮은 전력소모가 특징

- **IWDG/WWDG**
  - Independent/Windowed Watchdog
  - IWDG는 내부 RC 오실레이터를, WWDG는 메인 클럭을 통해 프로세서와 시스템의 오류를 감지하고 리셋을 수행

- **General Peripheral**
  - 일반 주변 장치, 기본적인 데이터 입출력, 제어 작업 수행
  - GPIO, Timer, UART, SPI, I2C, ADC, DAC, PWM, RTC, xWDG등 다양한 기능을 수행



KYUNG HEE UNIVERSITY

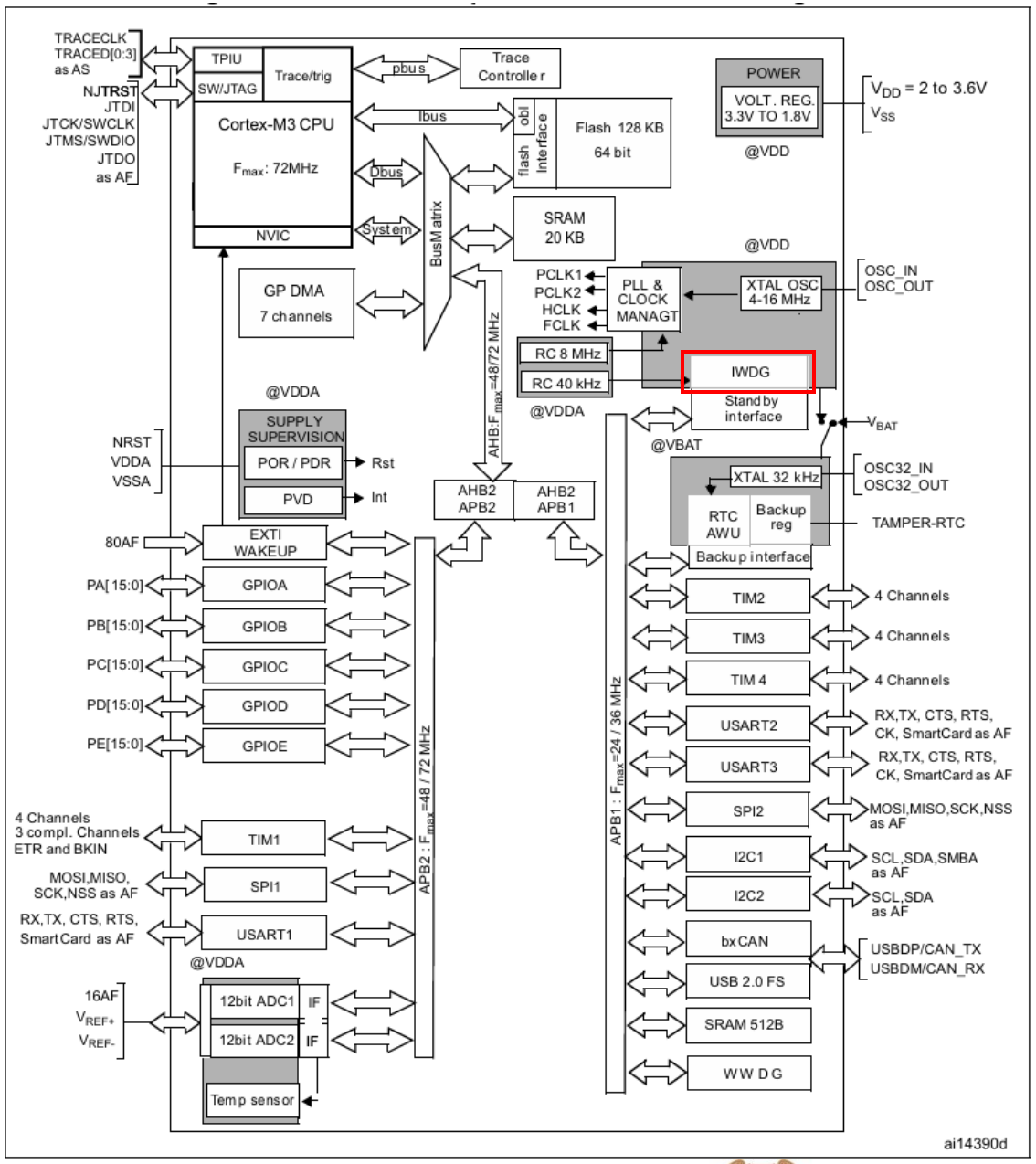
STM32F103RB

>> Structure

- **AHB/APB**
  - Advanced High-Performance Bus, Advanced Peripheral bus
  - AHB: 넓은 대역폭의 고속 데이터 전송 버스, 메모리와 DMA 컨트롤러, CPU 사이의 고속 데이터 통신 지원
  - APB: 저속 주변 장치(GPIO, UART, SPI, I2C 등) 사이의 통신 지원, 단순한 설계와 낮은 전력소모가 특징

- **IWDG/WWDG**
  - Independent/Windowed Watchdog
  - IWDG는 내부 RC 오실레이터를, WWDG는 메인 클럭을 통해 프로세서와 시스템의 오류를 감지하고 리셋을 수행

- **General Peripheral**
  - 일반 주변 장치, 기본적인 데이터 입출력, 제어 작업 수행
  - GPIO, Timer, UART, SPI, I2C, ADC, DAC, PWM, RTC, xWDG등 다양한 기능을 수행



KYUNG HEE UNIVERSITY

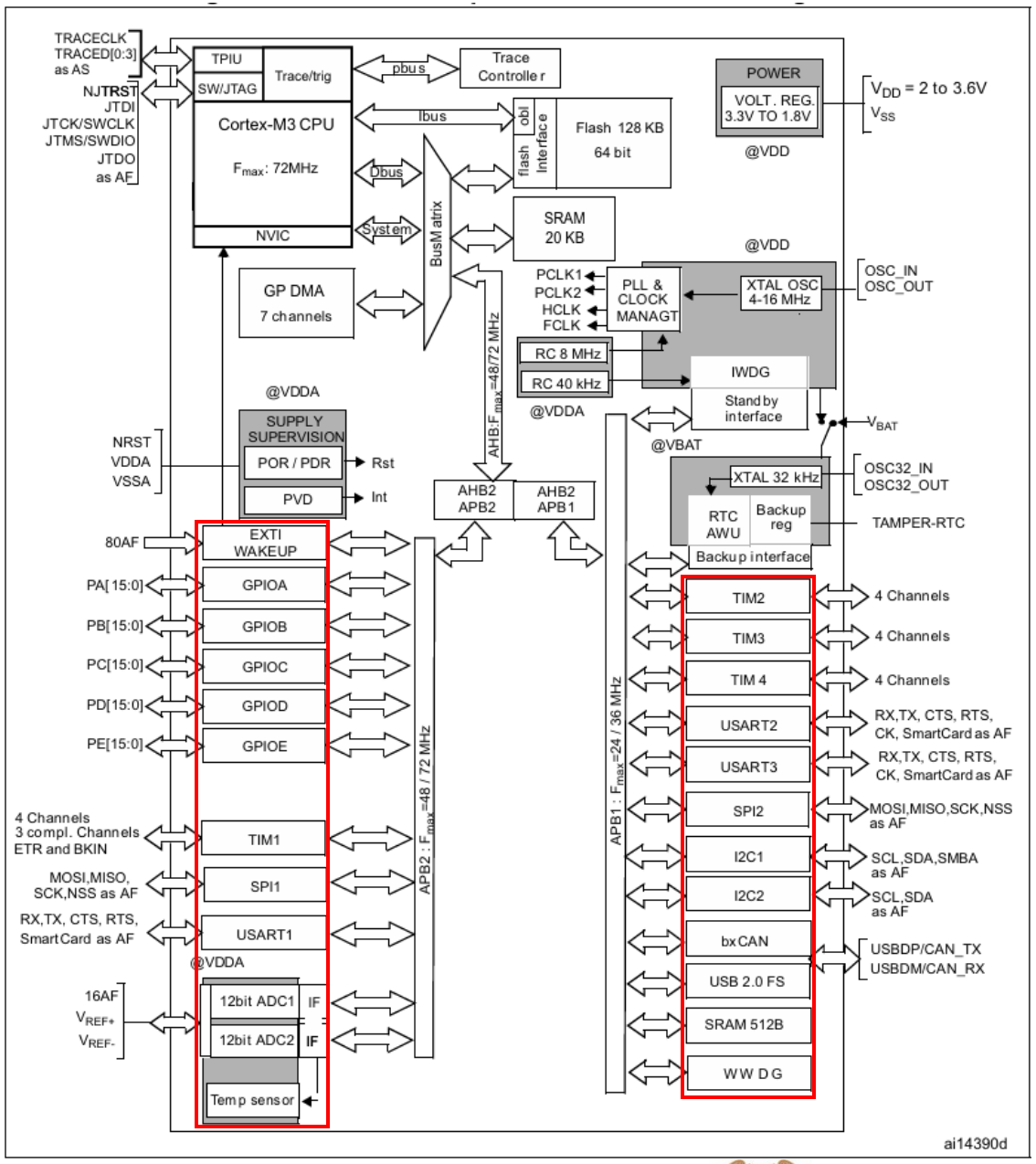
STM32F103RB

>> Structure

- **AHB/APB**
  - Advanced High-Performance Bus, Advanced Peripheral bus
  - AHB: 넓은 대역폭의 고속 데이터 전송 버스, 메모리와 DMA 컨트롤러, CPU 사이의 고속 데이터 통신 지원
  - APB: 저속 주변 장치(GPIO, UART, SPI, I2C 등) 사이의 통신 지원, 단순한 설계와 낮은 전력소모가 특징

- **IWDG/WWDG**
  - Independent/Windowed Watchdog
  - IWDG는 내부 RC 오실레이터를, WWDG는 메인 클럭을 통해 프로세서와 시스템의 오류를 감지하고 리셋을 수행

- **General Peripheral**
  - 일반 주변 장치, 기본적인 데이터 입출력, 제어 작업 수행
  - GPIO, Timer, UART, SPI, I2C, ADC, DAC, PWM, RTC, xWDG등 다양한 기능을 수행



KYUNG HEE UNIVERSITY

Hardware Abstraction Layer

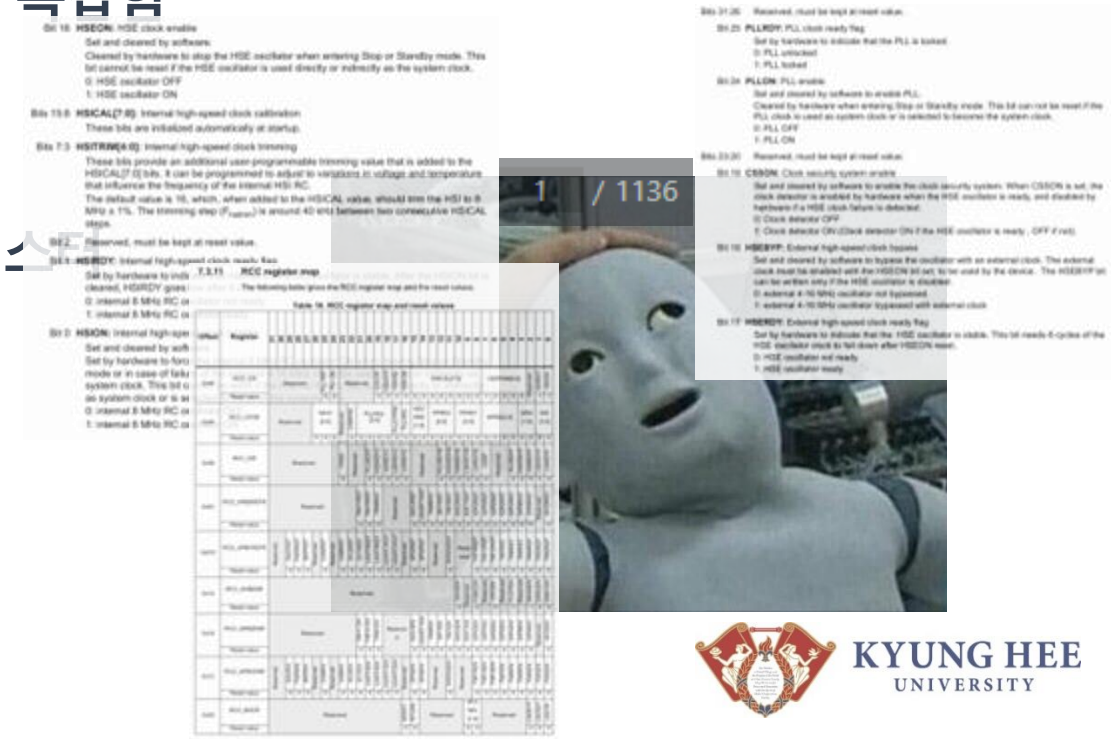
>> Hardware Abstraction Layer

- 프로세서 활용을 위해서는 하드웨어 레지스터를 직접 다뤄야함

- STM은 1000여 페이지의 Reference Manual을 통해 하드웨어 레지스터에 대한 모든 내용을 서술

- 하지만 사용자가 이를 모두 확인하여 설정하기에는 매우 복잡함

- Hardware Abstraction Layer, 하드웨어 레지스터 설정을 함수 호출로 추상화하여 사용자가 손쉽게 하드웨어 레지스터 설정을 가능케함

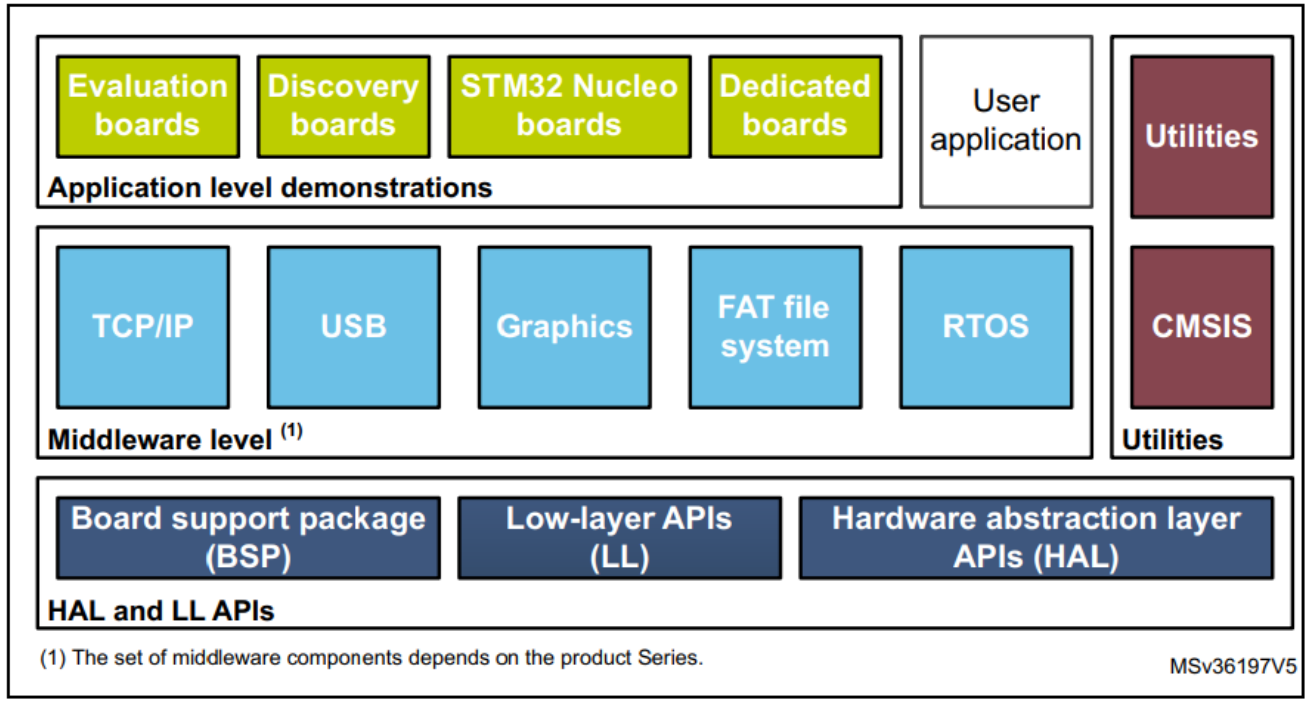


KYUNG HEE UNIVERSITY

Hardware Abstraction Layer

>> HAL & CMSIS

- ARM은 자사의 Cortex 시리즈에 대한 하드웨어 추상화 계층 구성을 위한 표준을 제공함 Cortex Microcontroller Software Interface Standard(CMSIS)
- HAL 드라이버는 CMSIS 기반으로 설계되어 고수준에서 장치 제어 기능을 효율적으로 제공함
- HAL 드라이버는 다양한 STM32 시리즈 프로세서간의 이식성을 높혀줌



Hardware Abstraction Layer

>> HAL & CMSIS

- CMSIS API에서 타겟 프로세서인 STM32F103xB 레지스터를 추상화

```
/*****************  Bit definition for RCC_APB1RSTR register  *****************/
#define RCC_APB1RSTR_TIM2RST_Pos            (0U)
#define RCC_APB1RSTR_TIM2RST_Msk            (0x1UL << RCC_APB1RSTR_TIM2RST_Pos) /*!< 0x00000001 */
#define RCC_APB1RSTR_TIM2RST                RCC_APB1RSTR_TIM2RST_Msk          /*!< Timer 2 reset */
#define RCC_APB1RSTR_TIM3RST_Pos            (1U)
#define RCC_APB1RSTR_TIM3RST_Msk            (0x1UL << RCC_APB1RSTR_TIM3RST_Pos) /*!< 0x00000002 */
#define RCC_APB1RSTR_TIM3RST                RCC_APB1RSTR_TIM3RST_Msk          /*!< Timer 3 reset */
#define RCC_APB1RSTR_WWDGRST_Pos            (11U)
#define RCC_APB1RSTR_WWDGRST_Msk            (0x1UL << RCC_APB1RSTR_WWDGRST_Pos) /*!< 0x00000800 */
#define RCC_APB1RSTR_WWDGRST                RCC_APB1RSTR_WWDGRST_Msk          /*!< Window Watchdog reset */
#define RCC_APB1RSTR_USART2RST_Pos          (17U)
#define RCC_APB1RSTR_USART2RST_Msk          (0x1UL << RCC_APB1RSTR_USART2RST_Pos) /*!< 0x00020000 */
#define RCC_APB1RSTR_USART2RST              RCC_APB1RSTR_USART2RST_Msk        /*!< USART 2 reset */
#define RCC_APB1RSTR_I2C1RST_Pos            (21U)
#define RCC_APB1RSTR_I2C1RST_Msk            (0x1UL << RCC_APB1RSTR_I2C1RST_Pos) /*!< 0x00200000 */
#define RCC_APB1RSTR_I2C1RST                RCC_APB1RSTR_I2C1RST_Msk          /*!< I2C 1 reset */
```

CMSIS: stm32f103xb.h

- HAL 드라이버는 CMSIS를 기반으로 하드웨어 설정을 구조체, 함수 형태로 표현하여 사용성 향상

- 또한 유사한 다른 프로세서로의 코드 재사용성, 이식성을 향상

```
void HAL_GPIO_Init(GPIO_TypeDef  *GPIOx, GPIO_InitTypeDef *GPIO_Init)
{
  uint32_t position = 0x00u;
  uint32_t ioposition;
  uint32_t iocurrent;
  uint32_t temp;
  uint32_t config = 0x00u;
  __IO uint32_t *configregister; /* Store the address of CRL or CRH register based on pin number */
  uint32_t registeroffset;       /* offset used during computation of CNF and MODE bits placement inside CRL or CRH register */
```

```
/* Includes ------------------------------
#include "stm32f1xx.h"
#include "Legacy/stm32_hal_legacy.h"
```

HAL Driver: stmf1xx_hal_gpio.h

- 최종적으로 사용자가 직접적으로 작성하는 main()에서는 HAL 드라이버를 통해 효율적으로 개발을 진행

```
while (1)
{
  /* USER CODE END WHILE */
HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
HAL_Delay(100);
  /* USER CODE BEGIN 3 */
}
```

main: main.c

KYUNG HEE UNIVERSITY

Debug

>> JTAG/SWD

- 프로세서와 하드웨어 회로를 구성하고, 요구사항에 맞는 동작을 위한 소스코드를 작성 및 업로드
- 하지만 일반적인 프로그래밍(C, C++, Python ...)과 달리 마이크로프로세서에는 터미널창이나 프로세스 모니터링 기능이 없음
- LED, LCD, 모터와 같은 별도의 출력 장치를 활용하여 동작을 간접적으로 확인
- 프로세서의 On-Chip Debug를 위해 JTAG/SWD 인터페이스를 활용함
- 오픈소스인 Open On-Chip Debugger(OpenOCD)와 STMicroelectroincs에서 제공하는 ST-Link를 활용

KYUNG HEE UNIVERSITY

Debug

>> ST-Link

Figure 3. Top layout

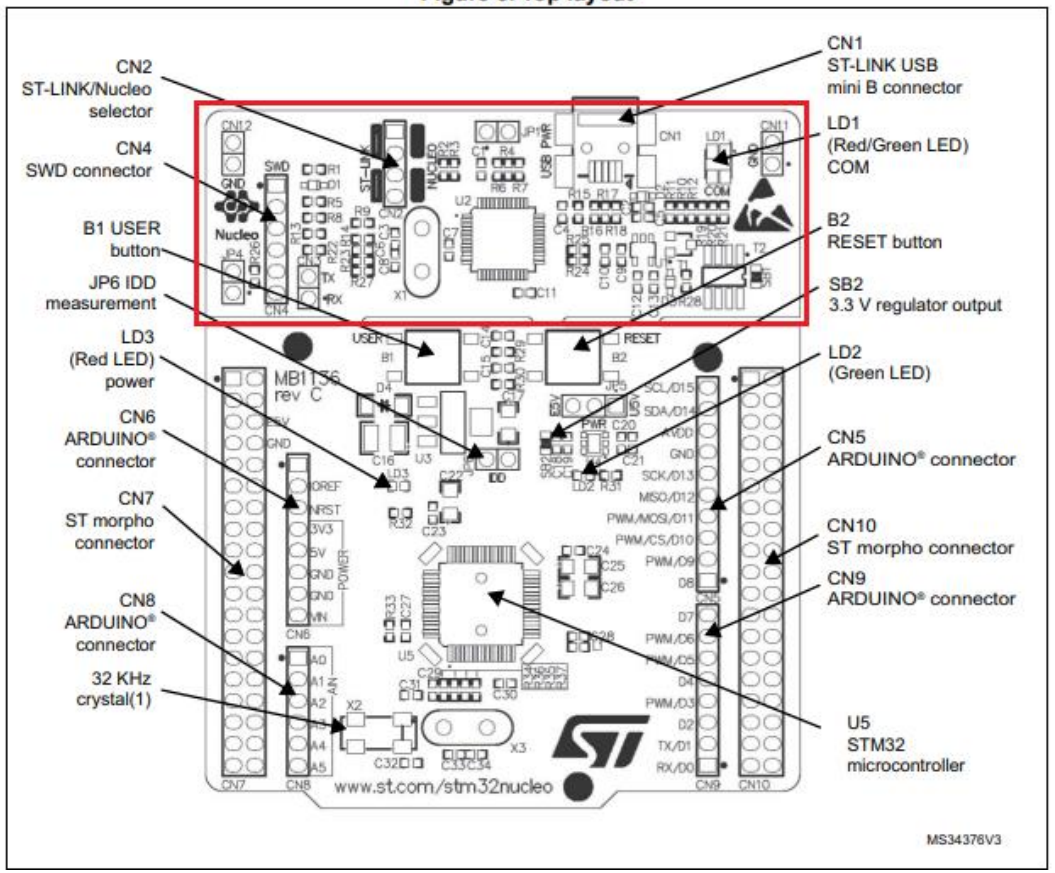
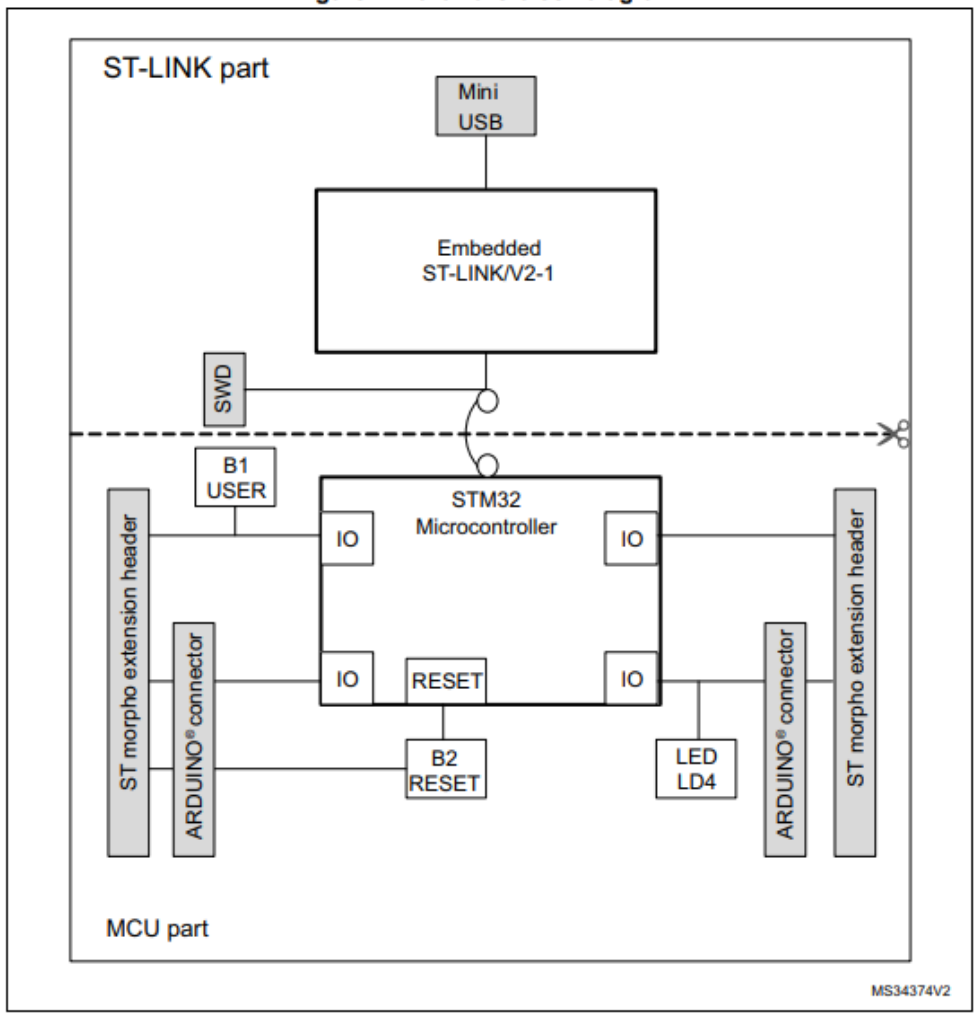


Figure 2. Hardware block diagram



Debug

>> GNU Debugger

File Edit Selection View Go Run Terminal Help

DISC

RUN A... Build & De

CMakeLists.txt | main.c | DISC.map | rlt_wrapper.h | rlt_wrapper.cpp | adam.h

Core > Src > main.c > main(void)

VARIABLES

Local
- mse = 6.75153351
- epoch = 3
- start = 17798052
- end = 28588459
- interv = 10790407

Global

Static: ./Core/Src/main.c

Registers

WATCH

CALL STACK

BREAKPOINTS

main.c Core\Src 144

CORTEX LIVE WATCH

Hint: Use & Enable "liveWatch" i...

XPERIPHERALS

- I2C3 @ 0x40005c00
- IWDG @ 0x40003000
- PWR @ 0x40007000
- RCC @ 0x40023800
  - CR @ 0x0 0x0f037983
  - PLLCFGR @ 0x4 0x07405408
  - CFGR @ 0x8 0x0000940a
  - CIR @ 0xc 0x00000000
  - AHB1RSTR @ 0x10 0x00000000
  - AHB2RSTR @ 0x14 0x00000000
  - AHB3RSTR @ 0x18 0x00000000
  - APB1RSTR @ 0x20 0x00000000
  - APB2RSTR @ 0x24 0x00000000
  - AHB1ENR @ 0x30 0x0010009f
  - AHB2ENR @ 0x34 0x00000080
  - AHB3ENR @ 0x38 0x00000000
  - APB1ENR @ 0x40 0x10228001

```c
95  int main(void)
125   MX_TIM2_Init();
126   /* USER CODE BEGIN 2 */
127   HAL_TIM_Base_Start(&htim2);
128   init_rlt();
129
130   float mse;
131   int epoch = 0;
132   uint32_t start, end, interv = 0;
133   /* USER CODE END 2 */
134
135   /* Infinite loop */
136   /* USER CODE BEGIN WHILE */
137   while (1)
138   {
139     /* USER CODE END WHILE */
140     MX_USB_HOST_Process();
141
142     /* USER CODE BEGIN 3 */
143     start = __HAL_TIM_GET_COUNTER(&htim2);
144     train(&mse);
145     end = __HAL_TIM_GET_COUNTER(&htim2);
146     // printf("!!!\n");
147     HAL_GPIO_TogglePin(LD3_GPIO_Port, LD3_Pin);
148
149     interv = end - start;
150     epoch++;
151     // HAL_Delay(100);
152   }
153   /* USER CODE END 3 */
154 }
155
156 /**
157   * @brief System Clock Configuration
158   * @retval None
159   */
160 void SystemClock_Config(void)
161 {
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | MEMORY | XRTOS | SERIAL MONITOR

Filter (e.g. text, !exclude, \escape)

```
Breakpoint 2, main () at C:/Users/Hyunwoo/Project/STM32/DISC/Core/Src/main.c:144
144         train(&mse);

Breakpoint 2, main () at C:/Users/Hyunwoo/Project/STM32/DISC/Core/Src/main.c:144
144         train(&mse);

Breakpoint 2, main () at C:/Users/Hyunwoo/Project/STM32/DISC/Core/Src/main.c:144
144         train(&mse);
```

OUTLINE

- hi2c1
- hi2s3
- hspi1
- htim2
- huart2
- PUTCHAR
- PUTCHAR
- HAL_UART_Transmit
- SystemClock_Config(void) declaration
- MX_GPIO_Init(void) declaration
- MX_I2C1_Init(void) declaration
- MX_I2S3_Init(void) declaration
- MX_SPI1_Init(void) declaration
- MX_USART2_UART_Init(void) declaration
- MX_TIM2_Init(void) declaration
- MX_USB_HOST_Process(void) declaration
- main(void)
- SystemClock_Config(void)
- MX_I2C1_Init(void)
- MX_I2S3_Init(void)
- MX_SPI1_Init(void)
- MX_TIM2_Init(void)
- MX_USART2_UART_Init(void)
- MX_GPIO_Init(void)
- Error_Handler(void)
- assert_failed(uint8_t *, uint32_t)

WORKSPACE

master | 0 0 | 0 | Build & Debug Microcontroller - ST-Link (DISC) | [Debug] | Build | [Debug] | [[Targets In Preset]] | [DISC] | [[Default]] | [[Default]] | Run CPack | [[Default]] | Run Workflow | Ln 143, Col 1 | Spaces: 2 | UTF-8 | CRLF | C | STM32

Debug

>> GNU Debugger

- GDB를 통해 디버깅을 진행하면 코드의 각 명령어 단위로 실행 흐름을 제어 할 수 있음
- 중단, 리셋, 함수 내부/외부 이동 등 다양한 실행 흐름 제어가 가능함
- 실시간으로 CPU 내부 레지스터와 글로벌/로컬 변수, Peripheral 레지스터 설정값등 다양한 정보 확인이 가능함
- 중단점(Breakpoint)을 통해 전체 실행 도중 원하는 지점에서 흐름을 멈추고 해당 지점 상태 확인이 가능함

Core > Src > main.c > main(void)

```c
95  int main(void)
125   MX_TIM2_Init();
126   /* USER CODE BEGIN 2 */
127   HAL_TIM_Base_Start(&htim2);
128   init_rlt();
129
130   float mse;
131   int epoch = 0;
132   uint32_t start, end, interv = 0;
133   /* USER CODE END 2 */
134
135   /* Infinite loop */
136   /* USER CODE BEGIN WHILE */
137   while (1)
138   {
139     /* USER CODE END WHILE */
140     MX_USB_HOST_Process();
141
142     /* USER CODE BEGIN 3 */
143     start = __HAL_TIM_GET_COUNTER(&htim2);
144     train(&mse);
145     end = __HAL_TIM_GET_COUNTER(&htim2);
146     // printf("!!!\n");
147     HAL_GPIO_TogglePin(LD3_GPIO_Port, LD3_Pin);
148
```

VARIABLES
- Local
  - mse = 6.75153351
  - epoch = 3
  - start = 17798052
  - end = 28588459
  - interv = 10790407
- Global
- Static: ./Core/Src/main.c
- Registers

CALL STACK

BREAKPOINTS
- main.c Core\Src 144

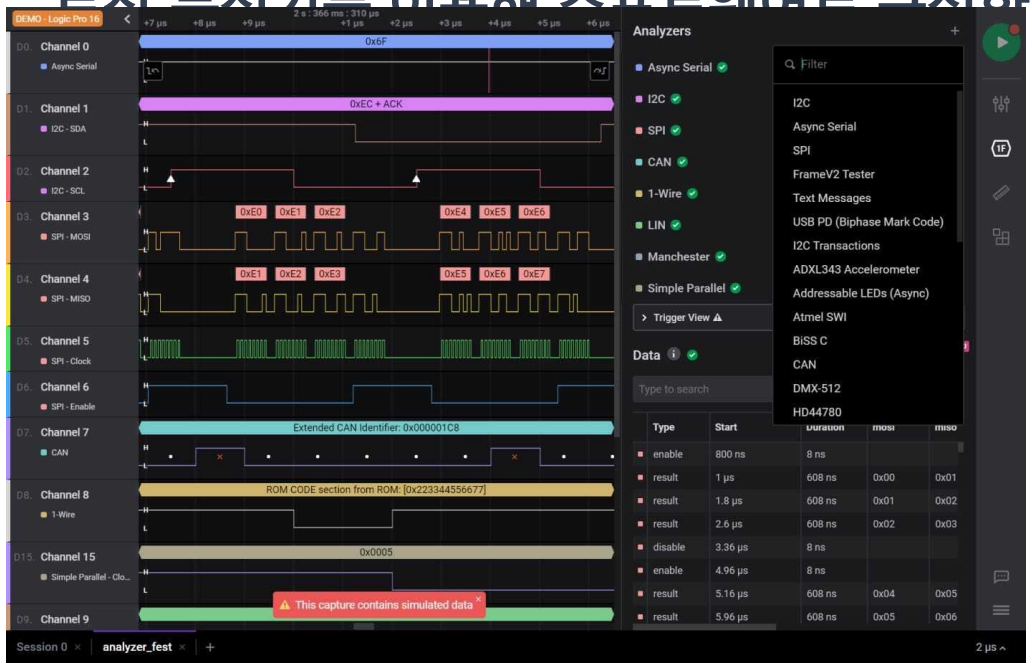
CORTEX LIVE WATCH
- Hint: Use & Enable "liveWatch" i...

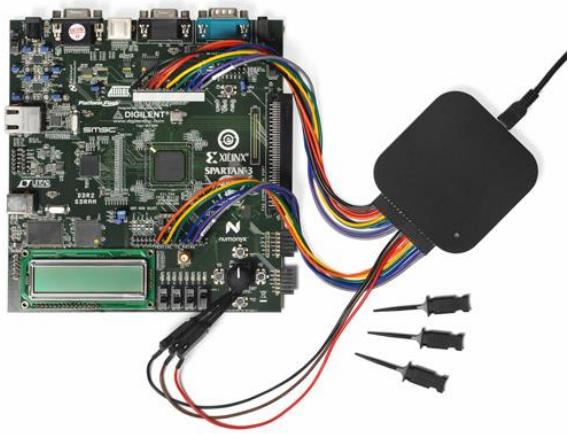
```c
143  start = __HAL_TIM_G
144  train(&mse);
145  end = __HAL_TIM_GET_
146  // printf("!!!\n");
147  HAL_GPIO_TogglePin(
148
149  interv = end - star
150  epoch++;
151  // HAL_Delay(100);
```

Debug

>> Logic Analyzer

- GDB는 소프트웨어 단계의 검증을 도와주지만 실제 하드웨어에 대한 검증은 불가하다
- 로직 분석기는 오실로스코프와 비슷하지만 이산적인 신호에 대한 분석을 도와줌
- 로직 분석기를 통해 전체 시스템의 입력, 출력, 통신 신호부터 단일 비트 단위까지의 신호 분석이 가능함
- 로직 분석기를 이용해 소프트웨어로 구성한 기능이 하드웨어적으로 올바르게 구현되었는지 검증이



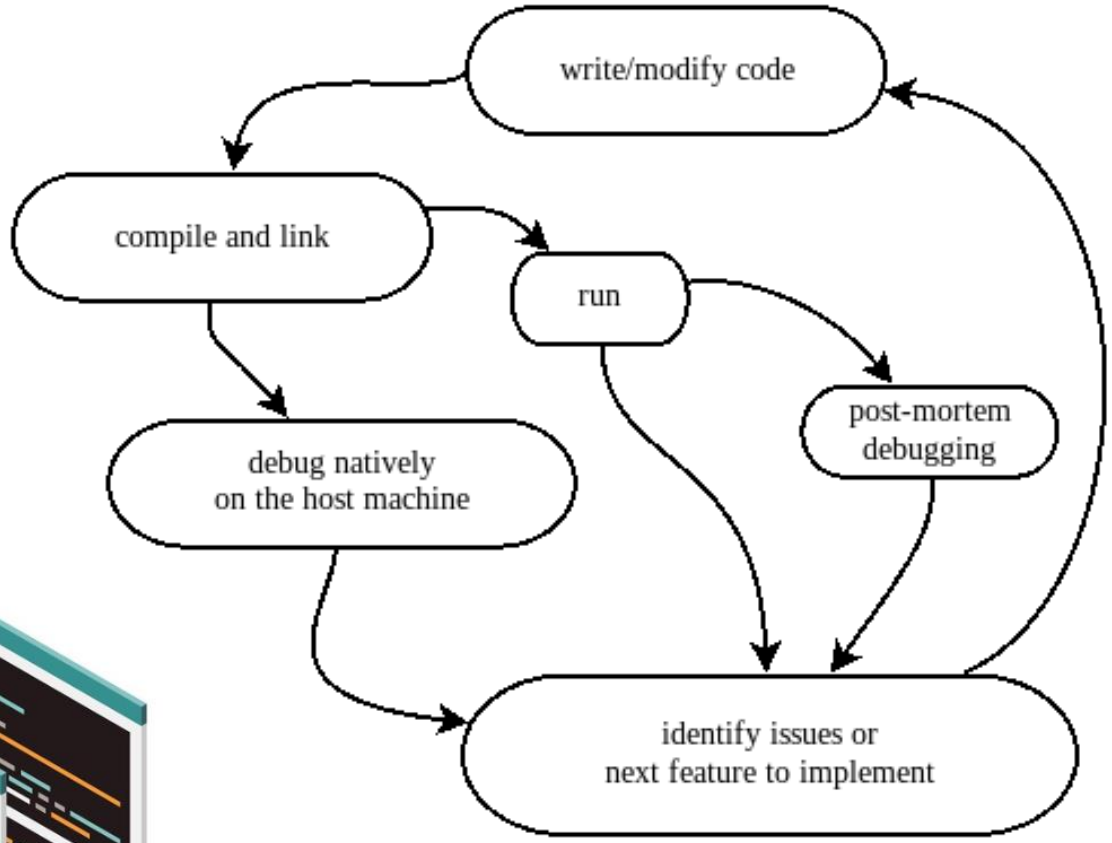


KYUNG HEE UNIVERSITY

임베디드/마이크로 프로세서 개발 과정

>> Workflow

- 요구 사항 분석 및 설계
- 하드웨어 설계
- 소프트웨어/펌웨어 개발
- 디버깅 및 테스트
- 시스템 통합 및 최적화
- 배포 및 유지 보수



Thank you

Department of Electronic Engineering,
Kyung Hee University, 1732, Deogyeong-Daero, Gih
eung-gu, Yongin-si, Gyeonggi-Do, 17104,
Republic of Korea
Web: https://khu-hri.weebly.com/

Human-Robot Interaction Laboratory

KYUNG HEE UNIVERSITY